EX. NO: 07. Date: 13/8/2026

Flow Control at Data Link Layer Using sliding window Protocol

Aim:

To write a program to implement flow control at the Data Link Layer using the sliding window protocol and to simulate the flow of frames from one node to another.

Algorithm:

* Start the program.
* Read the total number of frames to be sent (N), the sender's window size (W), and (optionally) a frame number to simulate as lost during transmission.
* Initialize a ~~frame number to 0~~ base = 0 (sequence number of the oldest unacknowledged frame) and next seq = 0 (sequence number of the next frame to be sent.
* Repeat while base < N:

a) Determine the current sender windows as the range [base, base + W-1].

b) Send all frames with sequence numbers from next seq up to (base + w - 1), Incrementing next seq after each frame, since the window restricts the sender to at

most N outstanding (unacknowledged) frames at any time.

c) If the lost frame lies within the frames just sent:

i) The receiver discards the lost frame and any subsequent out-of-order frames.

ii) The sender times out & retransmits all frames starting from the lost frame (Go-Back-N).

d) Else (no loss in this batch):

~~[i) The receiver accepts each frame any subsequent out-of-order frames.]~~

i) The receiver accepts each frame in order and sends a cumulative acknowledgement (ACK) for it.

ii) On receiving the Ack for the last frame sent, the sender slides its window forward: base = next seq.

e) Repeat step 4 until all N frames have been transmitted and acknowledged.

6) Stop the program.

Program:

```
def simulate(total_frames, window_size,
                                lost_frame):
    base = 0
    next_seq = 0
```

```
While base < total_frames:
    window_end = min(base + window_size - 1,
                                total_frames - 1)
    print("\n ----------- ")
    print(f"Sender Window : [Frame {base} - Frame
                                {window_end}]")
# Send all frames within the current window that
haven't been sent
    while next_seq < base + window_size and
                                next_seq < total_frames:
        print (f"Node A --> sending Frame {next_seq}")
        if next_seq == lost_frame:
            print (f"          (Frame {next_seq} Lost in
                                transit!)")
        next_seq += 1
    if lost_frame is not None and base <= lost_frame <
                                next_seq:
        # a frame in this batch was lost:
receiver discards it and
        # everything after it (GO-Back-N);
sender times out and
        # resends starting from the lost frame
        print (f"Node B --> Frame {lost_frame}
missing. Discarding out-of-order frames after it.")
        print(f"Node A --> Timeout! Resending
window starting from Frame {lost_frame}.")
        next_seq = lost_frame
        lost_frame = None  # loss already
                                handled; retransmission
        continue                succeeds.
```

no loss in this batch : receiver acknowledges every frame in order.

```
        for f in range(base, next_seq):
            print(f"Node B --> Received Frame {f}, Sending ACK {f}")
        print(f"Node A --> ACK {next_seq - 1} received, sliding window forward.")
        base = next_seq
    print("\n----------")
    print(f"All {total_frames} frames successfully transmitted from Node A to Node B.")

def main():
    print("=== SLIDING WINDOW PROTOCOL - FLOW CONTROL SIMULATION (GO-BACK-N) ===\n")
    total_frames = int(input("Enter total number of frames to send : "))
    window_size = int(input("Enter sender's window size : "))
    lost_frame = int(input(f"Enter frame number to simulate as LOST \n[0 to {total_frames - 1}, enter -1 for no loss]: "))
    if lost_frame == -1:
        lost_frame = None
    simulate(total_frames, window_size, lost_frame)

if __name__ == "__main__":
    main()
```

Output :

```
=== SLIDING WINDOW PROTOCOL - FLOW CONTROL
                SIMULATION (GO-BACK-N) ===

Enter total Number of frames to send : 8
Enter sender's window size           : 4
Enter frame number to simulate as LOST
(0 to 7, enter -1 for no loss)       : 5

- - - - - - - - - - - - - - - - - - - - -

Sender Window: [Frame 0 - Frame 3]
Node A --> Sending Frame 0
Node A --> Sending Frame 1
Node A --> Sending Frame 2
Node A --> Sending Frame 3
Node B --> Received Frame 0, Sending ACK 0
Node B --> Received Frame 1, Sending ACK 1
Node B --> Received Frame 2, Sending ACK 2
Node B --> Received Frame 3, Sending ACK 3
Node A --> ACK 3 received, sliding window
                              forward.

- - - - - - - - - - - - - - - - - - - - -

Sender Window: [Frame 4 - Frame 7]
Node A --> Sending Frame 4
Node A --> Sending Frame 5
        (Frame 5 LOST in transit!)
Node A --> Sending Frame 6
Node A --> Sending Frame 7
Node B --> Frame 5 missing. Discarding out-
                    of-order frames after it.
Node A --> Timeout! Resending window starting
                              from Fram 5.
```

Sender window : [Frame 4 - Frame 7].

Node A --> Sending Frame 5
Node A --> Sending Frame 6
Node A --> Sending Frame 7
Node B --> Received Frame 4, Sending ACK 4
Node B --> Received Frame 5, Sending ACK 5
Node B --> Received Frame 6, Sending ACK 6
Node B --> Received Frame 7, Sending ACK 7
Node A --> ACK 7 received. Sliding window forward.

All 8 frames successfully transmitted from Node A to Node B.

Result:

The program to implement flow control at the Data Link layer using the sliding window protocol was successfully written in Python, executed and verified by simulating the flow of frames between two nodes (Node A as sender & Node B as receiver).